

Software Requirement & Design Document (SRDD)

AI-Powered Subjective Exam Evaluation System (72-Hour Hackathon)

Project Lead: [Your Name]

Team Size: 3 Students

Duration: 72 Hours

Scope: MVP with 50 student PDFs, keyword-based grading, HITL review, output generation

Tech Stack: FastAPI, React, PostgreSQL, Gemini Vision, LLM Keyword Extraction

Deployment: Localhost → Vercel (post-hackathon)

1. Executive Summary & Problem Statement

Problem

Faculty manually grade subjective handwritten exams. This is time-consuming, inconsistent, and lacks structured feedback. Grading is biased toward handwriting quality, not answer content.

Solution

Automated keyword-based grading pipeline:

1. **OCR** extracts text from student handwritten PDFs
2. **Keyword Extraction** (LLM-assisted) identifies key concepts from answers
3. **Rubric Matching** awards marks based on keyword presence (ignoring grammar/spelling/handwriting)
4. **HITL Review** allows teachers to approve/modify ambiguous answers
5. **Output** generates individual PDFs + master spreadsheet

Scope (MVP)

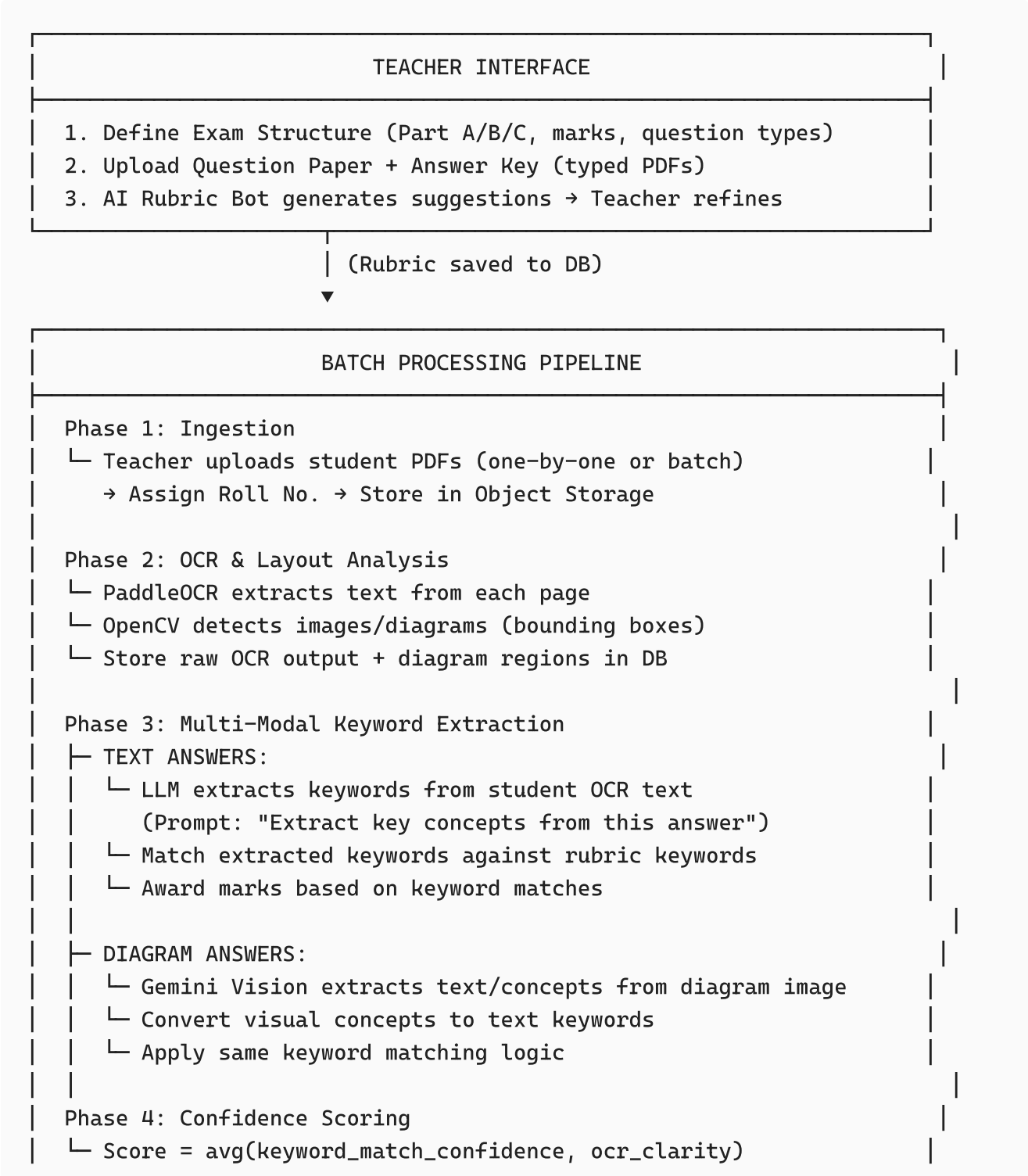
- ☒ 50 identical exam papers (same rubric, different student answers)
- ☒ Multi-page student PDFs (up to 35 pages)
- ☒ 3-part exam structure (Part A: 2-mark, Part B: 4-mark, Part C: 10-mark)
- ☒ Keyword-based evaluation (ignore grammar/spelling/handwriting)
- ☒ VLM-assisted diagram extraction (Gemini Vision)
- ☒ HITL dashboard (one-by-one review, approve/modify/add feedback)
- ☒ Individual grade PDFs (scanned answer + AI reasoning side-by-side)
- ☒ Master spreadsheet (scores, confidence, HITL status)

Out of Scope

- ❌ Double-blind privacy (TSID masking)
- ❌ Cheating detection (cosine similarity)
- ❌ Independent appeals workflow
- ❌ Multi-course management

2. System Architecture

2.1 High-Level Data Flow



└ If Score $\geq 0.7 \rightarrow$ Auto-pass, else $\rightarrow$ Flag for HITL

Phase 5: Database Storage

└ Save: extracted_keywords, matched_keywords, marks, confidence



HITL REVIEW DASHBOARD

└ Display: Student answer (OCR text + image crops)

└ Show: AI's extracted keywords, matched keywords, suggested mark

└ Teacher actions:

└ APPROVE (accept AI mark)

└ MODIFY (adjust mark + add reasoning)

└ FEEDBACK (add custom comment for student)

└ NEXT (move to next flagged answer)

└ (All grades finalized)

OUTPUT GENERATION

1. Individual Grade PDFs (per student):

└ Page-by-page scanned answer view (left pane)

└ AI reasoning for each question (right pane)

└ Overall score + feedback

2. Master Spreadsheet (CSV):

└ Roll No | Q1 Score | Q2 Score | ... | Total | Confidence

└ AI Flag (Yes/No) | Reviewed By

2.2 Architecture Components

Component	Technology	Purpose	Free Tier Limits
Frontend	React + HTML/JS	Teacher UI, rubric builder, HITL dashboard	Vercel (free)
Backend API	FastAPI (Python)	REST endpoints for all operations	Localhost $\rightarrow$ Vercel
OCR Engine	PaddleOCR (local)	Extract text from student PDFs	Free (local processing)
Image Detection	OpenCV (local)	Detect diagrams, crop regions	Free (local)

Component	Technology	Purpose	Free Tier Limits
Keyword Extraction	Gemini 2.0 Flash (free tier)	Extract keywords from OCR text	60 req/min limit
Diagram Text Extraction	Gemini Vision (free tier)	Extract text from diagram images	60 req/min limit
LLM Grading	Mistral AI (free tier)	Evaluate keywords against rubric	Free tier available
Database	PostgreSQL (Supabase free)	Store rubric, scores, OCR output	500MB limit
File Storage	Local filesystem (localhost)	Store uploaded PDFs, outputs	Vercel: 2.5GB ephemeral
Hosting	Vercel (free tier)	Deploy after hackathon	Free tier

3. Exam Structure & Rubric Format

3.1 University Exam Format (Your Specification)

EXAM STRUCTURE:

- Part A: 2-mark questions (ALL compulsory)
 - └ Answer: Max 25 words (short answer)
 - └ Example: "Define mitochondria" → Expect keywords: [mitochondria, energy, ATP]
- Part B: 4-mark questions (5 out of 7 to attempt)
 - └ Answer: Max 150 words (paragraph)
 - └ Example: "Explain photosynthesis process" → Expect keywords: [light, chlorophyll, glucose, CO2]
- Part C: 10-mark questions (3 out of 5 to attempt)
 - └ Answer: Max 500 words + diagrams allowed
 - └ Example: "Draw and label a plant cell" → Keywords: [nucleus, mitochondria, chloroplast, cell membrane]

Total: 16 marks (2×2 + 4×5 + 10×3 if all attempted)

3.2 Rubric Definition Format (UI-Based)

Teacher creates rubric via web UI:

```
{
  "exam_id": "physics_midterm_2024",
```

```

"subject": "Physics",
"total_marks": 16,
"exam_structure": [
  {
    "part": "A",
    "num_questions": 2,
    "marks_per_question": 2,
    "compulsory": true,
    "max_answer_length": 25,
    "questions": [
      {
        "q_number": "Q1A",
        "question_text": "Define velocity",
        "marks": 2,
        "keywords": ["velocity", "displacement", "time", "rate of
change"],
        "keyword_weights": {"velocity": 1.0, "displacement": 0.7, "time":
0.7, "rate of change": 0.8},
        "passing_threshold": 1,
        "grading_notes": "Must mention at least 'velocity' + one of the
others"
      }
    ]
  },
  {
    "part": "B",
    "num_questions": 7,
    "marks_per_question": 4,
    "compulsory": false,
    "attempt": 5,
    "max_answer_length": 150,
    "questions": [
      {
        "q_number": "Q1B",
        "question_text": "Explain photosynthesis",
        "marks": 4,
        "keywords": ["chlorophyll", "light", "glucose", "CO2", "H2O",
"oxygen"],
        "keyword_weights": {"chlorophyll": 1.0, "light": 1.0, "glucose":
1.0, "CO2": 0.8},
        "passing_threshold": 3,
        "grading_notes": "Deduct 1 mark per missing key concept"
      }
    ]
  }
]
}

```

3.3 AI Rubric Bot (ChatGPT/Claude-powered suggestion)

User Flow:

1. Teacher uploads Question Paper PDF (typed) + Answer Key PDF (handwritten)
2. System extracts text from both
3. LLM (Mistral/Claude) generates suggested rubric:

Prompt: "Given this question paper and answer key, suggest keywords and rubric structure"

Output: JSON rubric as above (auto-populated)

4. Teacher reviews & edits in UI
5. Saves to database

Key Advantage: Reduces teacher setup time from 30 min → 5 min

4. Keyword-Based Evaluation Engine

4.1 Why Keywords (Not Full LLM Grading)?

Traditional Approach:

Student writes: "Mitochondria are the powerhouse and produce ATP through oxidative phosphorylation"

LLM evaluates: "Correct answer, award 2/2 marks"

Problem: LLM also grades on grammar/completeness, not just content

Keyword Approach (Our System):

Student writes: "Mitochondria powerhouse produce ATP oxidashun phosphorlation"

OCR extracts: "Mitochondria powerhouse produce ATP oxidashun phosphorlation"

LLM keyword extraction: ["mitochondria", "powerhouse", "ATP", "oxidation", "phosphorylation"]

Rubric keywords: ["mitochondria", "ATP", "oxidation"] (min 2 required)

Match: 3/3 keywords found → 2/2 marks ✓

Advantage: Ignores spelling, grammar, handwriting quality

4.2 Keyword Extraction Pipeline

Step 1: Extract Keywords from Student Answer (LLM-assisted)

```
# Pseudo-code
```

```
def extract_keywords_from_answer(student_ocr_text, question_part,  
    question_number):
```

```
    """
```

```

    Input: Raw OCR text from student answer + question metadata
    Output: List of extracted keywords + confidence scores
    """

    prompt = f"""
Extract key concepts/keywords from this student answer.
Question Part: {question_part}
Question: {question_number}
Answer Text: {student_ocr_text}

Return ONLY a JSON array of keywords with confidence (0-1):
[
    {"keyword": "mitochondria", "confidence": 0.95}},
    {"keyword": "ATP", "confidence": 0.92}},
    ...
]
Ignore: spelling, grammar, punctuation. Focus on concepts.
    """

    response = gemini_flash.generate(prompt) # Gemini 2.0 Flash (60 req/min free)
    return parse_keywords_from_response(response)

```

Token Cost: ~150 tokens/answer (gemini-2.0-flash is optimized)

Step 2: Match Extracted Keywords Against Rubric

```

def evaluate_answer_keywords(extracted_keywords, rubric_keywords,
rubric_weights):
    """
    Input: Student keywords + Expected keywords from rubric
    Output: Matched keywords + score + confidence
    """

    matched = []
    score = 0

    for student_kw in extracted_keywords:
        best_match = find_fuzzy_match(student_kw["keyword"],
rubric_keywords)
        if best_match:
            matched.append({
                "student_keyword": student_kw["keyword"],
                "rubric_keyword": best_match,
                "confidence": student_kw["confidence"] *
rubric_weights[best_match]
            })
            score += rubric_weights[best_match]

    # Award marks based on score threshold
    if score >= rubric["passing_threshold"]:

```

```

        marks_awarded = rubric["marks"] * (score / len(rubric_keywords))
    else:
        marks_awarded = 0

    return {
        "matched_keywords": matched,
        "marks_awarded": marks_awarded,
        "confidence": avg([m["confidence"] for m in matched])
    }

```

Step 3: Handle Diagrams (Gemini Vision)

```

def extract_keywords_from_diagram(diagram_image_path, question_context):
    """
    Input: Cropped diagram image from student answer
    Output: Text description + extracted keywords
    """
    prompt = f"""
    Analyze this diagram and extract text/concepts shown.
    Question Context: {question_context}
    Return text labels and key concepts in this format:
    [
        {{"concept": "nucleus", "confidence": 0.95}},
        {{"concept": "mitochondria", "confidence": 0.92}},
        ...
    ]
    """

    image_base64 = encode_image(diagram_image_path)
    response = gemini_vision.generate(prompt, image_base64)  # Gemini Vision
    (free tier)
    return parse_concepts_from_response(response)

```

Token Cost: ~200-300 tokens/diagram (includes image encoding)

4.3 Confidence Scoring

```

def compute_confidence_score(evaluation_result):
    """
    Confidence = (keyword_match_confidence + ocr_clarity) / 2
    If confidence >= 0.7 → Auto-pass
    If confidence < 0.7 → Flag for HITL review
    """
    keyword_confidence = evaluation_result["confidence"]
    ocr_clarity = evaluation_result["ocr_clarity"]  # From OCR engine

    final_confidence = (keyword_confidence + ocr_clarity) / 2

```



```
    return {
        "confidence_score": final_confidence,
        "requires_hitl": final_confidence < 0.7,
        "reasoning": f"Keywords: {keyword_confidence:.2f}, OCR:
{ocr_clarity:.2f}"
    }
```

5. Data Schema & Database Design

5.1 PostgreSQL Tables + JSONB Columns

```
-- Exam metadata
CREATE TABLE exams (
    id SERIAL PRIMARY KEY,
    exam_name VARCHAR(255),
    subject VARCHAR(255),
    total_marks INT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Rubric (stored as JSONB for flexibility)
CREATE TABLE rubrics (
    id SERIAL PRIMARY KEY,
    exam_id INT REFERENCES exams(id),
    rubric_data JSONB NOT NULL, -- Full rubric JSON as defined in 3.2
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(exam_id)
);

-- Student PDF uploads
CREATE TABLE student_submissions (
    id SERIAL PRIMARY KEY,
    exam_id INT REFERENCES exams(id),
    roll_no VARCHAR(50),
    pdf_path VARCHAR(500), -- Path to uploaded PDF
    upload_timestamp TIMESTAMP DEFAULT NOW(),
    UNIQUE(exam_id, roll_no)
);

-- OCR output (raw text + layout info)
CREATE TABLE ocr_outputs (
    id SERIAL PRIMARY KEY,
    submission_id INT REFERENCES student_submissions(id),
    page_number INT,
    raw_text TEXT,
    has_diagram BOOLEAN,
```

```

    diagram_regions JSONB, -- [{x, y, width, height}, ...]
    ocr_clarity FLOAT, -- 0-1 confidence from PaddleOCR
    created_at TIMESTAMP DEFAULT NOW()
);

-- Question-level evaluation
CREATE TABLE evaluations (
    id SERIAL PRIMARY KEY,
    submission_id INT REFERENCES student_submissions(id),
    exam_id INT REFERENCES exams(id),
    question_id VARCHAR(50), -- e.g., "Q1A", "Q2B"
    extracted_keywords JSONB, -- [{keyword, confidence}, ...]
    matched_keywords JSONB, -- [{student_kw, rubric_kw, confidence}, ...]
    marks_awarded FLOAT,
    confidence_score FLOAT,
    requires_hitl BOOLEAN,
    ai_reasoning TEXT, -- Why these marks?
    diagram_extracted_text TEXT, -- If diagram present
    created_at TIMESTAMP DEFAULT NOW()
);

-- HITL Review (teacher edits)
CREATE TABLE hitl_reviews (
    id SERIAL PRIMARY KEY,
    evaluation_id INT REFERENCES evaluations(id),
    teacher_name VARCHAR(255),
    action VARCHAR(50), -- 'APPROVED', 'MODIFIED', 'REJECTED'
    final_marks FLOAT,
    teacher_feedback TEXT,
    reviewed_at TIMESTAMP DEFAULT NOW()
);

-- Master results (per student, final scores)
CREATE TABLE final_results (
    id SERIAL PRIMARY KEY,
    submission_id INT REFERENCES student_submissions(id),
    exam_id INT REFERENCES exams(id),
    roll_no VARCHAR(50),
    total_marks FLOAT,
    confidence_average FLOAT,
    num_hitl_reviews INT,
    reviewed_by VARCHAR(255),
    finalized_at TIMESTAMP DEFAULT NOW()
);

```

6. API Specifications (FastAPI)

6.1 Endpoints Overview

Endpoint	Method	Purpose	Auth
/exam/create	POST	Create new exam + structure	Teacher
/exam/{exam_id}/rubric-bot	POST	Generate rubric suggestions	Teacher
/exam/{exam_id}/rubric	PUT	Save/update rubric	Teacher
/exam/{exam_id}/submit-pdf	POST	Upload student PDF	Teacher
/exam/{exam_id}/process	POST	Start batch processing (OCR → evaluation)	Teacher
/exam/{exam_id}/results	GET	Get all results (auto-pass + HITL pending)	Teacher
/exam/{exam_id}/hitl-queue	GET	Get next HITL review item	Teacher
/exam/{exam_id}/hitl/{evaluation_id}	PUT	Submit HITL review decision	Teacher
/exam/{exam_id}/outputs	GET	Download individual PDFs + spreadsheet	Teacher

6.2 Key Endpoints (Detailed)

```
# 1. Create Exam
@app.post("/exam/create")
def create_exam(request: CreateExamRequest):
    """
    request = {
        "exam_name": "Physics Midterm",
        "subject": "Physics",
        "exam_structure": [
            {"part": "A", "num_questions": 2, "marks_per_question": 2, ...},
            ...
        ]
    }

    response = {
        "exam_id": 1,
        "status": "created"
    }
    """
    pass

# 2. Rubric Bot (AI-generated suggestions)
@app.post("/exam/{exam_id}/rubric-bot")
```

```

def generate_rubric_suggestions(exam_id: int, question_pdf: UploadFile,
answer_key_pdf: UploadFile):
    """
    1. Extract text from question PDF
    2. Extract text from answer key PDF
    3. Call LLM: "Generate rubric keywords + structure"
    4. Return suggested rubric JSON

    response = {
        "suggested_rubric": { full JSON rubric },
        "confidence": 0.85
    }
    """
    pass

# 3. Save Rubric
@app.put("/exam/{exam_id}/rubric")
def save_rubric(exam_id: int, rubric_data: dict):
    """
    Store rubric in DB
    """
    pass

# 4. Upload Student PDF
@app.post("/exam/{exam_id}/submit-pdf")
def upload_student_pdf(exam_id: int, roll_no: str, pdf_file: UploadFile):
    """
    1. Store PDF in filesystem (or S3-equivalent)
    2. Create student_submissions record
    3. Return submission_id
    """
    pass

# 5. Process (OCR + Evaluation)
@app.post("/exam/{exam_id}/process")
def process_exam_batch(exam_id: int):
    """
    For each submission in exam:
        1. OCR extract text (PaddleOCR)
        2. Detect diagrams (OpenCV)
        3. Extract keywords (Gemini Flash)
        4. Evaluate against rubric
        5. Compute confidence
        6. Store in DB

    Run asynchronously (background task)
    """
    pass

# 6. Get HITL Queue

```

```

@app.get("/exam/{exam_id}/hitl-queue")
def get_next_hitl_item(exam_id: int):
    """
    SELECT * FROM evaluations
    WHERE exam_id={exam_id} AND requires_hitl=true AND hitl_reviews.id IS
    NULL
    LIMIT 1

    response = {
        "evaluation_id": 42,
        "submission_id": 5,
        "roll_no": "A123",
        "question_id": "Q1B",
        "student_extracted_keywords": [...],
        "ai_reasoning": "...",
        "suggested_marks": 3.0,
        "confidence_score": 0.65,
        "ocr_text_preview": "Student wrote: ...",
        "diagram_text": "If diagram exists"
    }
    """
    pass

```

7. Submit HITL Review

```

@app.put("/exam/{exam_id}/hitl/{evaluation_id}")
def submit_hitl_review(exam_id: int, evaluation_id: int, review:
HitlReviewRequest):
    """
    review = {
        "action": "APPROVED" | "MODIFIED" | "REJECTED",
        "final_marks": 3.5,
        "teacher_feedback": "Good explanation but missing..."
    }

    1. Create hitl_reviews record
    2. Update evaluations table with final marks
    3. Return success
    """
    pass

```

8. Generate Outputs

```

@app.get("/exam/{exam_id}/outputs")
def generate_outputs(exam_id: int):
    """
    1. Generate individual Grade PDFs (one per student)
        - Left pane: Scanned answer pages
        - Right pane: AI reasoning + final marks per question
    2. Generate Master Spreadsheet (CSV)
        - Columns: Roll No | Q1A | Q1B | ... | Total | Confidence | HITL
    Status

```

```
3. Return download links or file streams
"""
pass
```

7. Frontend (React) Architecture

7.1 Components Structure

```
src/
├── components/
│   ├── ExamSetup/
│   │   ├── CreateExam.jsx
│   │   ├── ExamStructureBuilder.jsx
│   │   └── QuestionForm.jsx
│   ├── RubricBuilder/
│   │   ├── RubricBotChat.jsx (AI suggestions)
│   │   ├── RubricEditor.jsx
│   │   └── KeywordInput.jsx
│   ├── Upload/
│   │   ├── PDFUpload.jsx (drag-drop, one-by-one)
│   │   └── UploadStatus.jsx
│   ├── Processing/
│   │   ├── ProcessingDashboard.jsx
│   │   └── ProgressBar.jsx
│   ├── HITL/
│   │   ├── HitlQueue.jsx
│   │   ├── HitlReviewCard.jsx (display answer + reasoning)
│   │   └── HitlControls.jsx (approve/modify/feedback buttons)
│   └── Output/
│       ├── ResultsViewer.jsx
│       ├── DownloadOptions.jsx
│       └── SpreadsheetPreview.jsx
```

7.2 Key UI Screens (Wireframes)

Screen 1: Rubric Bot Chat

Step 1: Setup Rubric	
Upload Question Paper:	[Choose File]
Upload Answer Key:	[Choose File]
[Generate Rubric Suggestions]	

AI Bot: "I've analyzed your Q&A.
Here's a suggested rubric:"

Part A, Q1: Keywords [mitochondria,
ATP, oxidation]
Passing Threshold: 2/3

[Edit] [Confirm] [Regenerate]

Screen 2: HITL Review Card

STUDENT ANSWER (LEFT PANE)	AI EVALUATION (RIGHT PANE)
[Scanned PDF Page 1] Mitochondria are powerhouse... [Next Page] [Prev Page]	Question: Q1B Max Marks: 4 Extracted Keywords: ✓ chlorophyll (0.95) ✓ light (0.92) ✗ glucose (0) ✓ CO2 (0.88) Suggested Mark: 3.5/4 Confidence: 0.65 (HITL) Reasoning: "Missing key concept: glucose synthesis" Teacher Override: Final Mark: [3.5] / 4 Add Feedback: [Good effort. Explain...] [Approve] [Modify] [Next]

8. LLM Prompts (Token-Optimized)

8.1 Rubric Generation Prompt

SYSTEM: You are a university exam rubric expert. Generate marking rubrics from question papers.

USER INPUT:

Question Paper: [extracted text]

Answer Key: [extracted text]

Exam Structure: Part A (2 marks, 25 words), Part B (4 marks, 150 words), Part C (10 marks + diagrams)

PROMPT:

For each question, extract:

1. Keywords (concepts) expected in answer
2. Keyword weights (1.0 = essential, 0.7 = important, 0.5 = nice-to-have)
3. Minimum keywords needed to pass (passing_threshold)

Return ONLY valid JSON (no markdown, no explanation):

```
{
  "questions": [
    {
      "q_id": "Q1A",
      "keywords": ["keyword1", "keyword2"],
      "weights": {"keyword1": 1.0, "keyword2": 0.7},
      "passing_threshold": 1
    }
  ]
}
```

Token Budget: ~200 tokens (use gemini-2.0-flash for speed)

8.2 Keyword Extraction Prompt

SYSTEM: Extract key concepts from student answers. Ignore spelling, grammar, handwriting quality.

USER INPUT:

Student Answer: [OCR extracted text]

Question: [question text]

Question Type: [Part A / Part B / Part C]

PROMPT:

Extract only core concepts/keywords from this answer.

Return ONLY JSON (no explanation):

```
[
  {"keyword": "concept1", "confidence": 0.95},
  {"keyword": "concept2", "confidence": 0.88}
]
```

Token Budget: ~150 tokens per answer

8.3 Diagram Extraction Prompt (Gemini Vision)

SYSTEM: Analyze exam answer diagrams and extract text/concepts.

USER INPUT:

[Cropped diagram image]

Question Context: [question text]

PROMPT:

What text/labels/concepts are shown in this diagram?

Return ONLY JSON:

```
[
  {"concept": "label1", "confidence": 0.95},
  {"concept": "label2", "confidence": 0.88}
]
```

Token Budget: ~250 tokens per diagram (includes image encoding)

9. Development Plan (72-Hour Breakdown)

Phase 1: Setup & Infrastructure (Hours 1-4)

Tasks:

- Backend Setup** (2 hours)
 - FastAPI skeleton + project structure
 - PostgreSQL connection (Supabase free tier)
 - Environment variables (.env)
 - Basic error handling
- Frontend Setup** (2 hours)
 - React app creation (Vite for speed)
 - Folder structure + routing
 - Basic styling (Tailwind CSS)
 - API client setup (axios)

Git Strategy:

- `main` branch → production-ready
- `dev` branch → integration

- Feature branches: `feat/backend-setup` , `feat/frontend-setup`
-

Phase 2: Core Functionality (Hours 5-48)

Stream A: Backend (1 student)

Task 1 (Hours 5-12): OCR + Layout Analysis

- Integrate PaddleOCR (local)
- Integrate OpenCV (diagram detection)
- Save OCR output to DB
- Branch: `feat/ocr-engine`

Task 2 (Hours 13-24): Keyword Extraction + Evaluation

- Gemini Flash API integration (keyword extraction)
- Keyword matching logic
- Confidence scoring
- Branch: `feat/keyword-evaluation`

Task 3 (Hours 25-32): API Endpoints

- `/exam/create` , `/exam/{id}/rubric` , `/exam/{id}/submit-pdf` , `/exam/{id}/process`
- Error handling + logging
- Branch: `feat/api-endpoints`

Task 4 (Hours 33-40): HITL Review API

- `/exam/{id}/hitl-queue` , `/exam/{id}/hitl/{evaluation_id}`
- Database updates for HITL decisions
- Branch: `feat/hitl-api`

Task 5 (Hours 41-48): Output Generation

- Individual grade PDF generation (using reportlab or weasyprint)
- CSV spreadsheet generation
- Branch: `feat/output-generation`

Stream B: Frontend (1 student)

Task 1 (Hours 5-12): Exam Setup UI

- ExamStructure builder (Parts A/B/C selector)
- Question form (add questions dynamically)
- Branch: `feat/exam-setup-ui`

Task 2 (Hours 13-20): Rubric Bot Chat

- Chat interface for rubric suggestions
- File upload (question paper + answer key)
- Real-time rubric preview
- Branch: `feat/rubric-bot`

Task 3 (Hours 21-28): Student PDF Upload

- Drag-drop upload interface
- Roll number input + progress tracking
- Branch: `feat/upload-ui`

Task 4 (Hours 29-36): HITL Dashboard

- Display evaluation card (answer + reasoning)
- Approve/Modify/Feedback buttons
- Navigation (next/previous)
- Branch: `feat/hitl-dashboard`

Task 5 (Hours 37-44): Results Viewer

- Results table (all scores)
- Download options (PDF + CSV)
- Branch: `feat/results-ui`

Task 6 (Hours 45-48): Polish + Integration

- Fix styling, responsiveness
- Integration tests
- Branch: `feat/polish`

Stream C: Integration & LLM Prompts (1 student)

Task 1 (Hours 5-16): LLM Integration

- Gemini Flash (keyword extraction) setup + testing
- Mistral API setup (fallback)
- Prompt optimization
- Token budgeting
- Branch: `feat/llm-integration`

Task 2 (Hours 17-32): Rubric Bot Backend

- Implement rubric generation endpoint
- Test with sample question papers

- Refinement loop
- Branch: `feat/rubric-bot-backend`

Task 3 (Hours 33-48): Testing + Documentation

- Unit tests for keyword extraction
 - Integration tests for OCR → evaluation
 - README.md + API docs
 - Branch: `feat/testing`
-

Phase 3: HITL Review & Testing (Hours 49-60)

Parallel Testing:

- All 3 students test with 10 sample PDFs
- Fix bugs reported in HITL review
- Optimize token usage (track API costs)

Checkpoints:

- Hour 50: All endpoints working (manual testing)
 - Hour 55: HITL review functional
 - Hour 60: Output generation (PDFs + CSV) working
-

Phase 4: Production Run (Hours 61-72)

Final 12 Hours:

- Hour 61-64: Process all 50 student PDFs (batch run)
- Hour 65-70: Manual HITL review (teacher uses dashboard)
- Hour 71-72: Generate final outputs, GitHub push, documentation

Deliverables:

- ☒ 50 graded student PDFs
 - ☒ Master spreadsheet with all scores
 - ☒ Clean GitHub repo (well-documented)
 - ☒ Deployed on Vercel (demo link)
 - ☒ README with setup instructions
-

Git Workflow Strategy

```
# Day 1 Setup
git checkout -b dev
git checkout -b feat/backend-setup (Backend student)
git checkout -b feat/frontend-setup (Frontend student)
git checkout -b feat/llm-integration (Integration student)

# Daily Merges (End of each 8-hour shift)
git commit -am "Feature: OCR engine working"
git push origin feat/ocr-engine
git pull request → dev branch → code review (5 min) → merge

# Safe Parallel Work
- Backend works on `/app/services/ocr.py`
- Frontend works on `/src/components/ExamSetup.jsx`
- Integration works on `/app/services/llm.py`
→ No conflicts!

# Final Push (Hour 72)
git merge dev → main
git tag v1.0-hackathon
git push --all --tags
```

10. Free-Tier Tech Stack & API Limits

Service	Free Tier	Limit	How We Stay Safe
Gemini 2.0 Flash	Yes	60 req/min, 1500 req/day	Batch process, ~60 answers max
Gemini Vision	Yes	60 req/min	Only for diagrams (~10-15 per batch)
Mistral	Yes	Limited free tier	Fallback only, not primary
Supabase PostgreSQL	Yes	500MB, 2GB/month bandwidth	Efficient schema, minimal logging
Vercel	Yes	100GB bandwidth	Frontend only, API on localhost initially
PaddleOCR	Open source	Unlimited (local)	No API calls, runs on device
OpenCV	Open source	Unlimited (local)	Diagram detection local

Total Cost: \$0 (during hackathon, localhost only)

Post-Hackathon Cost (if deployed):

- Gemini API: ~\$0.30 per 1000 answers (low-volume academic use)
 - Vercel: Free tier (unless heavy traffic)
 - Supabase: Free tier (unless >500MB data)
-

11. Deployment & Hosting

Phase 1: Localhost (Hackathon)

```
# Backend (FastAPI)
pip install -r requirements.txt
python -m uvicorn app.main:app --reload

# Frontend (React)
npm install
npm run dev

# Database (PostgreSQL - Supabase free tier)
Connect via .env credentials
```

Phase 2: Vercel (Post-hackathon)

```
# Frontend → Vercel (automatic on git push)
vercel deploy

# Backend → Railway/Render (free tier)
# or Vercel serverless functions

# Database → Supabase cloud
# Just update .env with production credentials
```

12. Rubric Example (Reference)

```
{
  "exam_id": 1,
  "subject": "Biology",
  "exam_structure": [
    {
      "part": "A",
      "num_questions": 2,
      "marks_per_question": 2,
```

```

"compulsory": true,
"questions": [
  {
    "q_id": "Q1A",
    "question_text": "Define mitochondria",
    "marks": 2,
    "keywords": ["mitochondria", "organelle", "energy", "ATP"],
    "keyword_weights": {
      "mitochondria": 1.0,
      "organelle": 0.8,
      "energy": 0.9,
      "ATP": 1.0
    },
    "passing_threshold": 2,
    "grading_notes": "Must mention mitochondria + ATP"
  },
  {
    "q_id": "Q2A",
    "question_text": "Define photosynthesis",
    "marks": 2,
    "keywords": ["photosynthesis", "light", "glucose", "CO2",
"chlorophyll"],
    "keyword_weights": {
      "photosynthesis": 1.0,
      "light": 1.0,
      "glucose": 1.0,
      "CO2": 0.9,
      "chlorophyll": 0.8
    },
    "passing_threshold": 3,
    "grading_notes": "Must mention photosynthesis + light + glucose +
CO2"
  }
]
},
{
  "part": "B",
  "num_questions": 7,
  "marks_per_question": 4,
  "attempt": 5,
  "questions": [
    {
      "q_id": "Q1B",
      "question_text": "Explain the process of aerobic respiration",
      "marks": 4,
      "keywords": [
        "glycolysis", "Krebs cycle", "electron transport", "ATP",
        "glucose", "oxygen", "mitochondria", "pyruvate"
      ],
      "keyword_weights": {

```

```

        "glycolysis": 1.0,
        "Krebs cycle": 1.0,
        "electron transport": 1.0,
        "ATP": 0.9,
        "glucose": 0.8,
        "oxygen": 0.9,
        "mitochondria": 0.7,
        "pyruvate": 0.8
    },
    "passing_threshold": 5,
    "grading_notes": "Deduct 1 mark per missing major stage"
}
]
},
{
    "part": "C",
    "num_questions": 5,
    "marks_per_question": 10,
    "attempt": 3,
    "questions": [
        {
            "q_id": "Q1C",
            "question_text": "Draw and label a plant cell",
            "marks": 10,
            "has_diagram": true,
            "keywords": [
                "nucleus", "mitochondria", "chloroplast", "cell membrane",
                "cell wall", "vacuole", "endoplasmic reticulum", "Golgi"
            ],
            "keyword_weights": {
                "nucleus": 1.0,
                "mitochondria": 1.0,
                "chloroplast": 1.0,
                "cell membrane": 0.9,
                "cell wall": 0.9,
                "vacuole": 0.8,
                "endoplasmic reticulum": 0.7,
                "Golgi": 0.7
            },
            "passing_threshold": 6,
            "grading_notes": "Diagram evaluation: 1 mark per correctly labeled
organelle. Minimum 6 required."
        }
    ]
}
]
}
]
}

```


13. Error Handling & Edge Cases

Error	Handling
OCR fails (illegible handwriting)	Mark as low confidence (0.3), route to HITL
PDF too large or corrupted	Show error to teacher, ask to re-upload
Gemini API rate limit (60 req/min)	Queue requests, process in batches of 60
Keyword extraction returns empty	Confidence = 0, automatic HITL
Diagram not extracted properly	Fall back to OCR text, flag for HITL
Teacher modifies rubric mid-processing	Stop current batch, restart
Network failure during upload	Resume upload, validate file integrity

14. Success Criteria (Hackathon Judges)

✔ **Functional MVP:**

- 50 student PDFs → Graded automatically
- HITL dashboard used by teacher (at least 10 questions reviewed)
- Individual grade PDFs + master spreadsheet generated

✔ **Code Quality:**

- Clean, documented code on GitHub
- Proper error handling + logging
- API endpoints working (tested with Postman/curl)

✔ **Innovation:**

- Keyword-based approach (novel vs. full LLM grading)
- Rubric Bot (AI-assisted setup)
- VLM for diagrams (multi-modal)

✔ **Honest Scope:**

- README clearly states: "72-hour MVP, 50 students, no privacy layer, no cheating detection"
- Shows what was achieved vs. future work

15. Post-Hackathon Roadmap (v2 Features)

- ☐ Double-blind privacy (TSID masking)
 - ☐ Cheating detection (cosine similarity)
 - ☐ Independent appeals workflow
 - ☐ Multi-course management
 - ☐ Teacher analytics dashboard
 - ☐ Mobile app (React Native)
 - ☐ Batch processing improvements (async workers)
-

16. References & Resources

Libraries:

- PaddleOCR: <https://github.com/PaddlePaddle/PaddleOCR>
- OpenCV: <https://opencv.org/>
- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://react.dev/>

APIs:

- Gemini: <https://ai.google.dev/>
- Mistral: <https://mistral.ai/>
- Supabase: <https://supabase.com/>

Tools:

- Postman: <https://www.postman.com/> (API testing)
 - Vercel: <https://vercel.com/> (deployment)
-

End of SRDD

Last Updated: [Tonight]

Version: 1.0 (Hackathon)

Status: Ready for Development